Beyond Hallucination Detection: Measuring Consequential Uncertainty in LLM-Generated Code

Michael Jaquith
Independent Researcher
michaeljaquithresearch@gmail.com

April 2026

Abstract

Large language models generate code that looks uniformly confident. A correct call to `stripe.PaymentIntent.create()` and a deprecated call to `stripe.Charge.create()` are emitted with the same surface tone, even when the underlying token distribution is a near-50/50 split ($P = 0.52$ vs. 0.47). Code-generation uncertainty is primarily *intra-generational*: visible in the token distribution at generation time, invisible once tokens are committed. Sequence-level confidence signals — most notably the mean token probability p_{avg} of Spiess et al. [21] — read the same logprobs but aggregate them into one number per function; multi-sample methods measure output disagreement that simply does not occur for peaked code-token distributions, detecting at most 35% of HIGH-risk prompts on our benchmark at 5× cost.

We introduce ε, a per-token normalized Shannon entropy computed from the top-k log-probabilities returned by the generation API, AST-filtered to semantic tokens and max-pooled per function. At the function level ε is a recall-maximizing filter: on four models it flags 97.1% of HIGH API-integration prompts, with zero observed misses on the 200-entry evaluated set (recall-maximizing design) and precision = 0.272. The contribution is not the function-level flag. The contribution is that the same score localizes: of the ~ 98 tokens in a typical flagged function, only 10.5% exceed the flag threshold. ε narrows the developer's review surface from a whole function to an average of 10.4 tokens per function (10.5% of the function) — an 89% reduction in within-function review burden. This is the distinguishing claim. p_{avg}, at the same 1× API cost, is competitive on raw detection (65–90% vs. ε's 70–85% across four models) and in fact beats ε on two of four OpenAI models; what it cannot do is say *where in the function* to look. ε answers both.

The low function-level precision is the design. We pair the filter with a parallel LLM review loop that reclassifies each flagged entry as CLEARED or CONFIRMED, using ε's token-level attribution to direct the reviewer to the specific high-entropy tokens that drove the score. On 201 entries across three models, a context-primed reviewer clears 68.7% of flagged entries with zero late false positives. The contribution is the two-level view: function-level recall plus token-level localization, together with the first direct three-way empirical comparison of intra-generational, sequence-level, and inter-generational uncertainty signals on production-realistic code prompts.

1 Introduction

AI-assisted developers now generate code several times faster than they can read it. A single engineer working with a coding assistant produces pull requests that would previously have represented days of work, and the bottleneck shifts from writing to reviewing. Reviewers have finite attention and cannot inspect every line with equal care, yet the failure modes of modern code generation

concentrate in small, hard-to-spot places. Recent empirical work on deprecated APIs quantifies the scale of the problem: LLM-based code completion emits deprecated API calls at rates of roughly 25–38% on standard prompts and substantially more on legacy codebases, because the training corpus straddles library-version transitions and the model has no posterior knowledge of deprecation at inference time [24].

A reviewer facing AI-generated code has two problems, not one. The first is *which functions* to scrutinize; the second is *where within each function* to look. Existing uncertainty signals can answer the first question: static analyzers catch known-bad patterns; sequence-level confidence scores such as the mean token probability p_{avg} [21] produce one number per function; multi-sample methods [14, 3, 19] re-roll the same prompt and measure output disagreement. None of them answer the second. On a function the size of a non-trivial SDK call — roughly a hundred tokens — the difference between "flagged" and "flagged here, on this token, between these two alternatives" is the difference between directing a reviewer and asking them to re-read everything. Vasconcelos et al. [23] provide direct human-subjects evidence that this distinction matters in practice: in a controlled user study, highlighting tokens by their probability of being edited produced faster task completion and more targeted edits, while highlighting tokens by raw generation probability produced no benefit.

A concrete instance. A model asked to charge a credit card might emit `stripe.Charge.create(...)` — deprecated since 2019 — or `stripe.PaymentIntent.create(...)`, the current API. Both arrive in the developer's editor with the same syntactic fluency, the same confident indentation, and no visible marker distinguishing a well-chosen call from one the model guessed at. At the method-selector token, the two choices split the probability mass near 50/50 ($P = 0.52$ vs. 0.47); the output exposes none of this. Failures of AI code generation cluster at exactly these points: version-split APIs, renamed SDK methods, signature changes between library releases. The model is uncertain; the output is not.

Intra- versus inter-generational uncertainty. The gap between the internal split and a confident-looking output is not an accident of any particular model; it is structural. Multi-sample uncertainty quantification methods for LLMs are *inter-generational*: they sample the same prompt multiple times and measure disagreement across completions [14, 3, 19]. This works when the model's uncertainty surfaces as instability across re-rolls — factual questions, open-ended reasoning. Code is not such a task. A model with a near-tie prior on the deprecated branch samples that branch nearly every time; once the early tokens commit, the rest of the function is forced by context. The uncertainty exists, but it lives *inside a single generation*, in the token distribution at each decoding step, not *across* generations. We call this the *intra- versus inter-generational* distinction.

ε, and two levels at which it operates. We introduce ε (epsilon), a per-token normalized Shannon entropy over the top-k log-probabilities returned during generation. After AST-based filtering to keep only semantically meaningful tokens (attribute accesses, keyword arguments, import targets — not whitespace, comments, or cosmetic identifiers), we aggregate via max-pooling to obtain a single score per function. The function-level score is a recall-maximizing filter: at threshold $\varepsilon \geq 0.30$ it flags 97% of HIGH prompts with zero observed misses on our evaluated benchmark set. A reviewer asked "which functions need scrutiny" gets a clear answer; at that scale, so does p_{avg}. But the function-level view discards most of what ε knows. Of the ~ 98 tokens in a typical flagged function, only 10.5% exceed the flag threshold. The same score that flags the function also tells the reviewer that the relevant uncertainty lives in the method selector on line 4, not in the parameter block on lines 7–12. ε is a two-level signal: a function-level filter that agrees with p_{avg} on "which,"

and a token-level localizer that no sequence-level aggregate can produce.

Figure 1 stages the full problem on Stripe Scenario A (§3.6): what the reviewer sees is a single fluent line of code; what the model nearly emitted instead is a near-tie with a different, currently-supported API; and where ε directs the review is to the few tokens within the 98-token function where that hesitation lives. The same score that flags the function names the tokens that drove the flag.

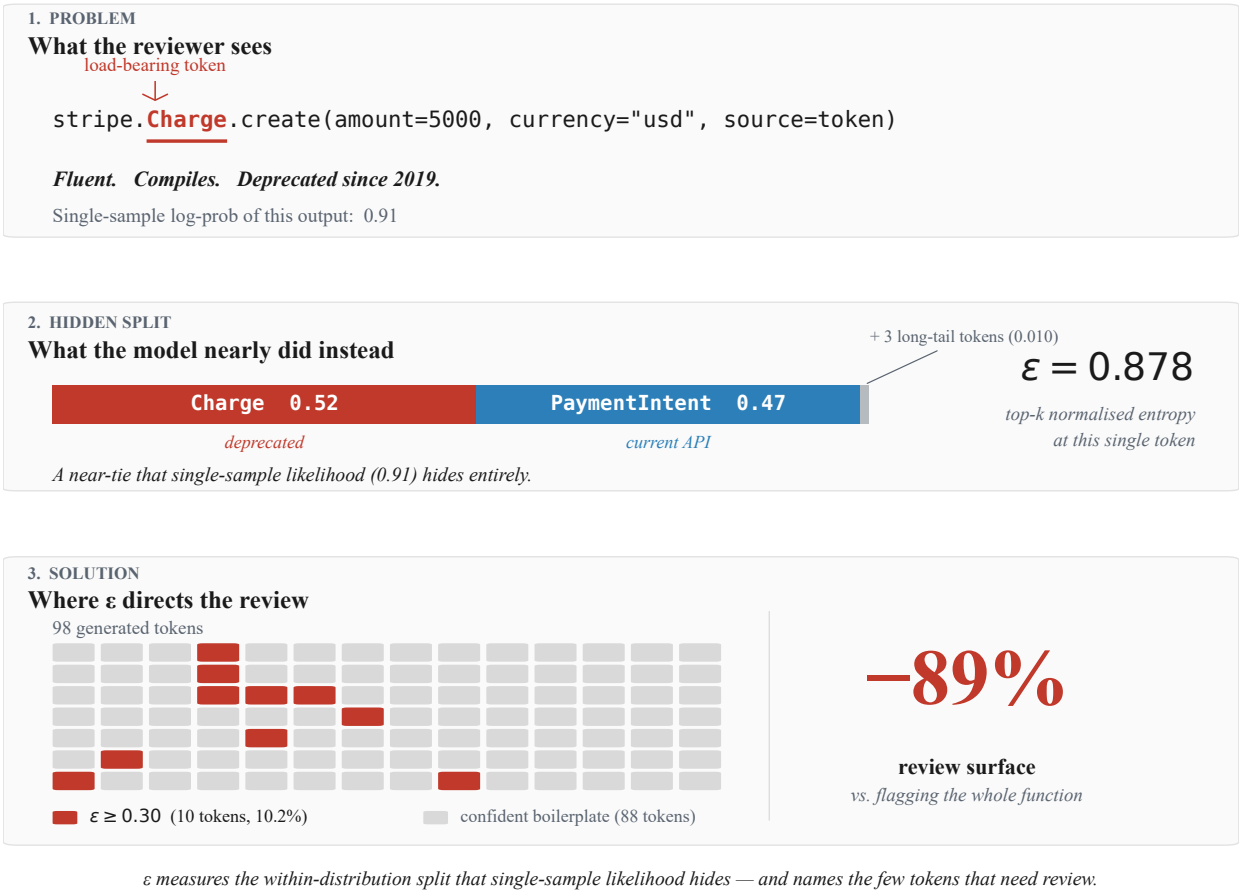

Figure 1: The problem, the hidden distribution, and ε's response, on Stripe Scenario A (GPT-4o). **Left:** the reviewer sees a fluent, compiling call to `stripe.Charge.create(...)` — deprecated since 2019 — with a single-sample log-probability of 0.91. **Centre:** at the method-selector token, the top-k distribution is a near-tie between `Charge` ($P = 0.52$, deprecated) and `PaymentIntent` ($P = 0.47$, current); the top-k-normalized entropy at this token is $\varepsilon = 0.878$. **Right:** of the 98 semantic tokens in the function, 10 (10.2%) exceed the flag threshold; the remaining 88 are confident boilerplate. ε narrows the within-function review surface by 89% and points the reviewer at the specific tokens that drove the flag. The paper-level claim in one image: single-sample likelihood hides the within-distribution split that ε reads, and ε names the few tokens that need review.

A three-way comparison. Because the function-level claim is not distinguishing, the comparison in this paper is between *signals*, not just detection rates. We benchmark ε against two alternatives at the same 1× API cost and one at 5× on the same production-realistic prompts across four models. At 1×, the sequence-level mean token probability p_{avg} [21] is competitive with ε on raw function-level detection (§5); what it cannot do is identify the specific token at which the model was uncertain. ε

produces both the function-level score and the token-level attribution. At 5× cost, multi-sample methods at $N = 5$, $T = 0.7$ detect 10–35% of HIGH prompts: the 52/47 hesitation does not surface as output variation, so inter-generational signals are structurally blind to it. UQLM [3], the closest packaged off-the-shelf tool, detects 5% on the same benchmark.

The two-stage system. A 27%-precision filter is not meant to be used in isolation. Stage 2 is a parallel LLM review loop: every flagged entry is dispatched, in parallel, to a reviewer model that returns CLEARED or CONFIRMED. The reviewer is not asked to re-read the whole function. It is asked to examine the specific high-ε tokens — on Stripe Scenario A, the handful of tokens whose ε exceeds the flag threshold. At current API prices this costs fractions of a cent per entry and completes in seconds. The token-level attribution is not decoration on the filter; it is the mechanism that makes the reviewer fast and precise. On 201 entries across three models, a context-primed reviewer clears 68.7% of flagged entries with zero late false positives.

Contributions.

1. The intra/inter-generational distinction, formalized and empirically verified: ε reaches 85% detection on GPT-4o and DeepSeek V3 against 10–35% for direct multi-sample measurement and 5% for UQLM on the same benchmark.
2. Token-level localization as ε's distinguishing claim: the function-level flag agrees with p_{avg}; the token-level attribution is what no sequence-level signal can produce.
3. A two-stage filter-plus-review architecture: 94% end-to-end precision on 201 entries across three models, with the reviewer directed to high-ε tokens.
4. The first direct three-way empirical comparison — ε vs. p_{avg} vs. multi-sample — on production-realistic version-split prompts, isolating where each signal fires and where it does not.

We use the labels HIGH and LOW throughout: HIGH prompts are API/version-split tasks across Stripe, OpenAI, SQLAlchemy, FastAPI, and Pydantic where an incorrect choice causes functional failure; LOW prompts are pure-logic tasks (sort, palindrome, Fibonacci, two-sum) where any correct algorithm is functionally equivalent. A detailed calibration table appears in §4.4.

Implementation and benchmark are at `https://github.com/michaeljaquith/epsilon-code`.

2 Related Work

Four adjacent lines of work inform the design of ε: empirical studies of deprecated API usage (which motivate the problem), token-level and sequence-level calibration of LLMs for code (which share the underlying logprob signal), multi-sample uncertainty quantification (which is the dominant alternative and the closest point of comparison), and adaptive decoding (which uses the same per-token entropy signal for a different purpose). We review each in turn, closing with static analysis.

2.1 Deprecated and version-split API usage

Wang et al. [24] present the first systematic study of deprecated API usage in LLM-based code completion, evaluating seven models on 145 API mappings drawn from eight popular Python libraries across 28,125 completion prompts. They measure API Usage Plausibility (AUP) and Deprecated Usage Rate (DUR), finding that deprecated-API emission is prevalent across all evaluated models; the root cause they identify is that deprecated API usages are present in training data while deprecation

status itself is not reliably propagated to inference time. Their mitigations (REPLACEAPI and INSERTPROMPT) operate at the prompt or post-generation level and require a known deprecation mapping.

Their finding establishes that the problem we address is common in practice rather than a corner case: on version-split APIs and renamed SDK methods, models routinely commit to a deprecated branch without surface indication. Our work is complementary. Wang et al. measure *how often* LLMs emit deprecated code and diagnose *why* the output looks confident; ε measures the *signal at generation time* that distinguishes a confidently-chosen branch from an internally-contested one, and — crucially for the review loop — localizes that signal to a specific token rather than producing only a function-level score.

2.2 Token-level and sequence-level calibration

Spiess et al. [21] study calibration and correctness of language models for code, computing several confidence signals (including the sequence-level mean token probability, which we refer to throughout as p_{avg}) and evaluating whether they correlate with functional correctness on HumanEval and related benchmarks. They find that generative code models are not well-calibrated out of the box and that Platt scaling can recover calibration in settings where labelled correctness data is available. The p_{avg} signal — the exponential of the mean log-probability over all generated tokens — is derived from the same raw logprob data that ε uses, and is the closest signal-level comparator for the function-level part of ε's output.

Four differences between that setup and ours deserve emphasis. First, *aggregation*: p_{avg} takes a sequence-level mean of per-token probabilities; ε takes a max after AST filtering, so a single high-entropy API-selector token is not diluted by long confident stretches. Second, *filtering*: ε restricts the input set to semantically meaningful tokens (attribute accesses, keyword arguments, import targets), while p_{avg} averages over every token including whitespace and cosmetic names. Third, *prompt regime*: Spiess et al. evaluate on HumanEval and similar benchmarks where correctness is well-defined by unit tests; we evaluate on version-split API prompts where the ground truth is ambiguous enough to require LLM judging (§6.1). Fourth, *output granularity*: p_{avg} produces one scalar per function and is silent on *which token* carried the uncertainty. This fourth distinction is the one that matters for the review loop (§5). Because p_{avg} and ε operate on the same underlying logprobs at the same $1\times$ API cost, a direct head-to-head comparison is possible; we report it in §5.

Two concurrent code-specific studies further establish that token-level uncertainty localizes to the same regions ε targets. Sharma and David [19] adapt entropy and mutual information to code generation and show, using symbolic execution as ground truth, that uncertainty-based abstention reduces incorrect outputs to near zero; their finding grounds the core premise of ε in prior empirical evidence without requiring symbolic execution at inference time.

Gros and Devanbu [7] address the same sub-function localization question via a different route: they train a separate supervisor probe on edited-region labels to estimate, in a white-box fashion, which lines of generated code a developer is likely to edit, reporting calibrated Brier Skill Scores around 0.2 on edited lines. ε differs along three axes. It reads the generator's own top-k distribution at the same API call, with no auxiliary model required; it requires no labeled edit data to train against; and it is integrated into a recall-first filter plus stage-2 reviewer pipeline rather than offered as a standalone calibrated signal. Their work and ours agree that sub-function localization is feasible and informative; they differ on whether the localization comes from a trained supervisor or directly from the generator's exposed distribution.

The closest prior work on AST-grounded token-level confidence for code is ASTrust [16], which maps token-level model confidence to AST syntax categories (e.g. `Identifier`, `Keyword`, `Literal`) and validates the resulting heat-map / sequence / graph visualizations across 12 LLMs and human studies as an interpretability artifact for code review. ε and ASTrust read the same underlying per-token signal, and ASTrust establishes that grouping that signal by AST structure is useful for developers. The differentiation is in how the AST is used and how the signal is consumed downstream. ASTrust *groups* tokens by AST category for visualization; ε *filters* on AST role to remove Type 1 cosmetic uncertainty (declaration positions: function names, parameter names, local variable bindings) before scoring, so that the surviving high-entropy tokens are concentrated on attribute accesses, keyword arguments, and import targets where the consequential / implementation distinction (§3.2) makes them actionable. The Type 2/Type 3 taxonomy is what converts a localized signal into a routing decision: a token is forwarded to a stage-2 reviewer because of where it falls in the taxonomy, not because of where it falls in the AST. ASTrust's contribution is the explanatory overlay; ε's contribution is the routing layer of an end-to-end filter-plus-reviewer system that directs a downstream model to the specific tokens that drove the flag.

Earlier token-level calibration studies (Kadavath et al. [12]; Varshney et al. [22]) establish that LLMs are reasonably calibrated at the token level for factual questions and that selective generation can exploit this. Gupta et al. [8] prove that sequence-level aggregation of token probabilities suffers from length bias—longer outputs are systematically over- or under-penalised by mean logprob—and show that token-level deferral rules substantially outperform naïve sequence-level means; this provides a principled explanation for why p_{avg} is a challenging baseline to beat despite sharing the same raw logprob signal. Santilli et al. [18] further demonstrate that length bias compromises *evaluation* of uncertainty methods across eight signals and four models, validating our choice to compare ε and p_{avg} at matched operating points rather than by raw AUC. Our work extends that line to code, where AST filtering is what makes per-token entropy actionable rather than dominated by cosmetic variance. Conformal prediction approaches [17, 1] provide set-valued predictions with coverage guarantees but do not localize which token within a generation is uncertain; TECP [25] applies split conformal prediction directly to token-entropy scores, providing per-output coverage guarantees that could in future work be used to calibrate the ε threshold with statistical guarantees. Uncertainty propagation through multi-stage ML pipelines is studied in adjacent domains [11].

2.3 Multi-sample and behavioural-diversity methods

Inter-generational methods sample the same prompt multiple times and measure disagreement among outputs. Kuhn et al. [14] introduce semantic entropy: multiple completions are clustered by semantic equivalence (via natural language inference), and cross-cluster entropy provides a hallucination signal for factual QA. UQLM [3] packages semantic negentropy and related signals as an open-source library; we include it here as the closest off-the-shelf tool and report its performance in §5 in one sentence. Beyond these, a growing line of work [19] applies stronger notions of equivalence — up to and including symbolic execution or semantic-equivalence checks on generated code — to improve behavioural consistency detection specifically for code outputs.

These methods are structurally limited for the task we study: intra-generational uncertainty does not surface as output variation when the token distribution is peaked, as it typically is for code. A model can be internally torn between two API calls and emit one of them unanimously across re-rolls; we provide the empirical instance in §5.3. A stronger equivalence check improves precision on Type 3 (implementation-approach) false positives, where two semantically equivalent implementations would be conflated by output-level diversity alone; but it cannot recover a signal that never varies

across samples in the first place. Song et al. [20] sharpen the equivalence check further by computing depth-bounded AST subtree distributions across multiple samples and comparing them via Jensen-Shannon divergence and a Structural Cross-Entropy ratio — a more principled, AST-driven diversity measure than NLI clustering for cases where the output *does* vary across re-rolls. It shares the structural blindness of multi-sample methods for our setting (a confidently-emitted deprecated API is structurally identical across samples by construction, so no AST-derived diversity measure will fire), but is the appropriate tool when sampled outputs do diverge structurally. The intra/inter distinction is not a claim that inter-generational methods are badly implemented, only that they are measuring a different quantity.

2.4 Generation-time intervention and adaptive decoding

AdaDec [9] reads the same per-token Shannon entropy that ε reads, but uses it at decoding time rather than review time: when the entropy at the current step exceeds a learned threshold, the decoder pauses and triggers a lookahead re-ranking step over candidate continuations. On HumanEval and MBPP across model sizes from 0.6B to 8B, AdaDec reports up to 15.5pp improvement in Pass@1 over greedy decoding while pausing on 3.95–11.88% of tokens — the same order of magnitude as ε's flagged-token rate of 10.5% (§3.6), which is not a coincidence: both identify the same set of decision points. The signal is the same as ours; the action is different. AdaDec improves the model's own output at generation time; ε surfaces residual uncertainty *after* the model has committed, for routing to downstream review. The two approaches are compatible: an AdaDec-enhanced generator still produces a token distribution at each step, and ε operates on whatever distribution the final decoder emits. We do not evaluate a combined pipeline but expect ε values to be lower on average after AdaDec-style intervention, with residual high-ε tokens concentrating on consequential decisions that even a lookahead re-rank could not resolve. Consequently, applying AdaDec upstream of our pipeline is expected to reduce average ε values but not eliminate high-entropy API-selector tokens, which remain the primary target of the review loop.

2.5 Entropy-guided refinement and self-repair

A separate line of work uses uncertainty signals to trigger iterative repair rather than review. Self-Refine [15] establishes the canonical pattern: a single model generates, critiques its own output, and refines iteratively, achieving $\approx 20\%$ absolute improvement across diverse tasks without supervision. The refinement loop is unconditional—every generation is sent back regardless of confidence.

Conditional entropy-triggered variants tighten this by gating second-pass work on measured uncertainty. Correa and de Matos [6] compute Shannon entropy over top-k alternatives at each step, trigger refinement when entropy, perplexity, or low-confidence token counts exceed learned thresholds, and feed an uncertainty report back to the originating model; they report 16 pp accuracy gains at one-third the cost of reasoning-class models. ERGO [13] applies the same principle to multi-turn generation, resetting the prompt context when entropy spikes sharply across turns.

Our review loop shares the entropy-triggered gating with these methods but differs in two ways. First, the second-pass agent is a separate reviewer with domain-specific prompting, not a self-critique call to the originating model; this removes the correlation between generator failure and reviewer failure. Second, the review is over *code-specific risk*—the flagged token and its alternatives, not the full response—which concentrates reviewer attention and enables the precision improvement from 27% to 94% (§5).

2.6 Static analysis, linters, and execution-based evaluation

Static analysis and linter rules catch *known*-bad patterns for specific libraries once deprecation has been encoded into a ruleset. These approaches are complementary to ε and represent the floor of detection: when a rule exists, a linter will catch the problem whether or not the model hesitated. ε operates in the gap — it fires where the model itself was uncertain, which includes version splits not yet in any ruleset (a library updated last week, a half-migrated codebase) and internal renames that static analysis has no fact to test against. Execution-based code-generation benchmarks [5, 2, 10] measure whether the output passes test cases, not whether the model was confident; a silent dependency update can turn a fully-passing generation into a regression that no test in the current suite exercises. Token-level uncertainty for code is studied prior to this work — ASTrust [16] groups token-level confidence by AST category for interpretability, AdaDec [9] reads the same per-token signal at decoding time, and Vasconcelos et al. [23] demonstrate user benefit from token-level highlighting in IDEs. To our knowledge, no prior work packages this signal as the routing layer of an end-to-end filter-plus-reviewer pipeline that directs a downstream model to the specific tokens that drove the flag; the closest pipeline-level analogues are CodeT [4] and related self-consistency approaches, which are inter-generational by construction, and post-generation repair with uncertainty-based ranking [26], which operates on already-generated candidates.

3 ε: Definition, Construction, and Localization

3.1 Per-token entropy

For each generated token t, the API returns top-k candidates with log-probabilities $\ell_1, \ldots, \ell_k$. Let $p_j = \exp(\ell_j)/\sum_j \exp(\ell_j)$ be the renormalized distribution. Define

$$\varepsilon_t \;=\; -\frac{1}{\log k}\sum_{j=1}^{k} p_j \log p_j \;\in\; [0,1]. \tag{1}$$

Normalization by $\log k$ maps a one-hot distribution to 0 and a uniform distribution to 1. We use $k = 5$ (the OpenAI cap), which covers the dominant mass for code tokens; tails beyond top-5 are negligible for the binary flag decision. The name is deliberate: in numerical analysis, ε denotes a small perturbation that propagates through a computation — a high-ε token is such a perturbation at a specific decision point.

3.2 Three types of code-generation uncertainty

Raw per-token entropy, applied naively, produces too many false positives to be useful: the bulk of high-entropy tokens in generated code are not risk indicators. Understanding which ones are, and why, is what drives the filtering design in the rest of this section. Empirical analysis of our calibration benchmark reveals three categorically distinct types of token-level uncertainty in generated code.

Type 1 — Cosmetic (naming). Uncertainty about what to *call* something (`sort_list` vs. `sort_numbers`). ε reaches 0.7–0.9, but any choice is correct. Dominates LOW prompts: the top flag on 9 of 10 LOW prompts was a function or parameter name on the first line.

Type 2 — Consequential (API/library decisions). Uncertainty about which *API or library* to use, e.g. `stripe.Charge.create()` vs. `stripe.PaymentIntent.create()` at the attribute token. This is the target signal.

Type 3 — Implementation-approach. Uncertainty among multiple correct algorithms (recursive vs. iterative Fibonacci, for-loop vs. comprehension, brute vs. hash-based two-sum). *The Type 2 / Type 3 distinction is definitional, not structural.* A given high-ε token is classified as Type 3 precisely when all of the competing continuations produce functionally correct code; it is classified as Type 2 when at least one continuation is wrong. The underlying decoder state can look identical in both cases — a 0.5-peaked top-2 distribution at a decision point — and ε has no way, from the token distribution alone, to know which label applies. The label is a property of the external world (are all branches correct?) and is only visible via ground truth. Consequently the residual Type 3 false-positive rate is irreducible at the signal level: a perfect classifier in a world where both `list.sort()` and `sorted(list)` are correct cannot help firing on the choice. What the system can do is not *suppress* the fire but *resolve* it: Type 3 false positives are fast for the stage-2 reviewer to clear because both alternatives pass inspection.

Type	Label	Source	ε fires?	Risk
1	Cosmetic	Naming choices (`sort_list` vs. `sort_numbers`)	Yes (0.7–0.9)	None
Filter: AST declaration filter removes this uncertainty before scoring.				
2	Consequential	API / library decisions (`Charge` vs. `PaymentIntent`)	Yes (0.5–0.9)	**High**
Target signal: this is what the system is designed to catch.				
3	Implementation	Algorithm structure where all branches are correct (recursive vs. iterative)	Yes (0.3–0.7)	None
Definitional Type 3: classified Type 3 because all continuations work.				

Table 1: The three-type uncertainty taxonomy. Type 2 vs. Type 3 is a property of the target world (are the alternatives all correct?), not of the decoder state. ε cannot distinguish them from the token distribution alone; the stage-2 reviewer resolves them by inspecting the alternatives.

With this taxonomy in hand, the filter design follows directly: remove Type 1 syntactically, leave Type 3 as a known floor that the reviewer is cheap to clear, and isolate Type 2.

3.3 AST filtering

We eliminate Type 1 via the generated code's AST. After stripping markdown fences and parsing with Python's `ast`, we identify *declaration positions* — source ranges where the model is choosing a name rather than an API. **Filtered:** function/method names, parameter names (`ast.arg`), local variable names (`ast.Name` in `Store`). **Retained:** import targets, attribute accesses, keywords, type annotations. Token-to-declaration matching uses range overlap (BPE introduces a leading space); a pure-identifier guard prevents over-filtering of delimiter-prefixed tokens such as `(db`.

3.4 Aggregation: function-level max

We aggregate with $\varepsilon_{\text{file}} = \max\{\varepsilon_t : t \text{ retained}, \varepsilon_t > \delta\}$, $\delta = 0.30$. The compound alternative $1 - \prod(1 - \varepsilon_t)$ converges to 1.0 for any non-trivial function. Across all 90 runs of the benchmark (3×30), max and mean produced identical fire/no-fire decisions at every threshold; we prefer max because *it preserves the token identity*, not just the function-level score. A mean would give a reviewer one number; a max plus the argmax gives a reviewer a specific token to look at. The contrast with the sequence-level mean token probability baseline (p_{avg}) is deliberate: averaging over

Configuration	low FP	high det.
Naive token entropy (no filtering)	100%	100%
+ Whitespace / comment filtering	100%	100%
+ System prompt naming conventions	70%	95%
+ AST declaration filtering	60%	80%
+ Fence-format stripping	50%	80%
Residual (Type 3 only)	50%	—

Table 2: Successive filtering stages on 10 LOW and 20 HIGH prompts (GPT-4o, fresh staging run). LOW FP is the fraction of pure-logic prompts that fired; HIGH det. is the fraction of API-version-split prompts that fired. The AST declaration filter trades a 15pp drop in HIGH detection (95% → 80%) for a 10pp reduction in LOW FP (70% → 60%): three HIGH prompts whose peak-ε token was a function name (not an API selector) fall below the flag threshold once declarations are removed. This is correct behaviour — those functions were uncertain about *naming*, not about *which API to call* — but it is an honest trade-off rather than a free win. Residual 50% LOW FP is Type 3 (irreducible); stages 1–2 use a bare system prompt without naming-convention instructions.

every token dilutes a single uncertain decision and, more importantly, discards the information that would tell a reviewer where to look. We revisit this choice empirically in §5.

3.5 Thresholds and tiers

Three tiers: COMPLETE ($\varepsilon < 0.30$), FLAGGED ($0.30 \leq \varepsilon < 0.95$), PAUSED ($\varepsilon \geq 0.95$). The load-bearing boundary is the binary COMPLETE vs. FLAGGED+: everything FLAGGED+ is forwarded to the stage-2 reviewer. PAUSED is a higher-urgency signal within the flagged region. These are UX signals, not calibrated probabilities — the FLAGGED empirical failure rate is $\sim 27\%$ (§4).

3.6 Token-level focus: the distinguishing claim

The function-level flag rate answers the question "which functions need scrutiny." For HIGH prompts it says "essentially all of them"; a reviewer given only the function-level signal is told little more than "re-read the function." The token-level view is where ε becomes actionable.

Let N_f be the number of retained (AST-filtered, non-cosmetic) tokens in a function, and let $N_f^{\geq\tau}$ be the count of those tokens with $\varepsilon_t \geq \tau$. The *focus ratio* at threshold τ is

$$\rho_\tau(f) \;=\; N_f^{\geq\tau} \,/\, N_f. \tag{2}$$

$\rho_{0.30}$ is the fraction of semantic tokens that exceed the flag threshold. We report ρ at the flag threshold because that is the boundary at which the stage-2 reviewer is directed to inspect a token; the PAUSED function-level threshold ($\varepsilon \geq 0.95$) is near-empty at the token level on this corpus (only 2 tokens across 144 entries cross it, so the rate is not a useful data point).

The numbers in Table 3 are the central quantitative claim of this paper. A developer reviewing an AI-generated function does not face a 98-token problem. They face an $\sim$ 10-token problem at the flag threshold — the tokens ε identified as high-entropy. The function-level flag rate of 97% is not a problem to be defended against; it is the entry point to a localization step that no sequence-level aggregate produces and that prior token-level visualizations [16, 23] provide only as standalone artifacts rather than as routing into a review loop.

Set	Fn-level flagged	$\varepsilon > 0.30$ (tokens)	Tokens@flag per fn
HIGH ($n = 104$)	97.1%	10.5%	10.4
LOW ($n = 40$)	90.0%	13.1%	10.4

Table 3: Function-level flagging vs. token-level focus across four models (GPT-4o, GPT-4o-mini, GPT-4-turbo, DeepSeek V3), 30 prompts, 144 entries total. The function-level flag rate is near-saturating on HIGH prompts; the token-level focus narrows the within-function review surface to 10.5% of the function at the flag threshold. Average function size is ~ 98 retained semantic tokens. Token review burden reduction at the flag threshold: 89%.

Figure 2 shows the token-level ε trajectory on Stripe Scenario A (GPT-4o). Of 98 retained tokens, 10 exceed the flag threshold. The highest-ε tokens cluster on the `.Ch` method selector on `stripe` (the Charge-vs-PaymentIntent decision) and one immediately adjacent token. The remaining tokens are confident parameter construction, imports, and return-value handling. A reviewer given this figure does not re-read the function; they look at the highlighted tokens.

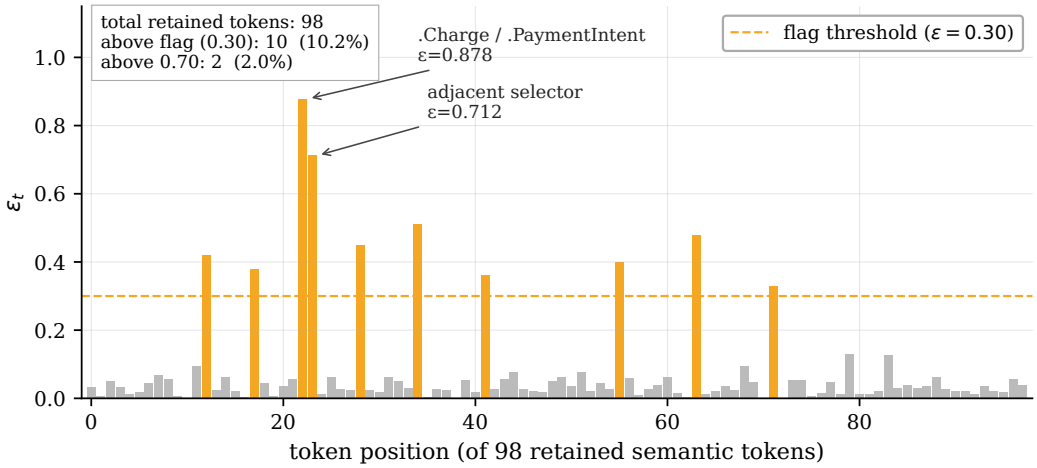

Figure 2: Per-token ε trajectory, Stripe Scenario A (GPT-4o, 98 retained tokens). The flag threshold (0.30) is dashed. Ten tokens exceed the flag threshold; the peak localizes to the method-selector decision between `Charge` and `PaymentIntent`. The rest of the function is confident boilerplate. ε does not just flag the function — it names the tokens that drove the flag.

3.7 Multi-model calibration at a glance

ε at the FLAGGED threshold fires on HIGH prompts much more often than on LOW prompts. Across four models (GPT-4o, GPT-4o-mini, GPT-4-turbo, DeepSeek V3), ε achieves 70–85% HIGH detection at 50–70% LOW false-positive rate. A detailed calibration table appears in §4.4.

3.8 Adaptive ensemble

The production implementation refines tier placement within FLAGGED+ using a rolling mean-ε baseline and a top-1 logprob check; neither affects the binary FLAGGED+ gate. Conformal calibration is left to future work. The current thresholds ($\varepsilon \geq 0.30$ for FLAGGED, $\varepsilon \geq 0.95$ for PAUSED) are

calibrated empirically on GPT-4o and inherited by other models without formal coverage guarantees. A natural defense path is split conformal prediction over per-token entropy scores in the style of TECP [25], which would convert the flag threshold into a coverage-controlled decision (e.g., 90% coverage of the tail of token-entropy values on a held-out calibration set). The current evaluation supports the thresholds as UX signals with measured failure rates; statistical guarantees are a complementary calibration step, not a replacement for the empirical evidence.

4 Evaluation

We evaluate ε along four axes: the token-level focus claim introduced in §3.6, qualitative scenarios that surface specific failure modes, a multi-model calibration benchmark, and a precision/recall sweep on a 200-entry combined set. The review loop results follow in §4.7.

4.1 Token-level focus across models and prompt categories

Table 3 (in §3.6) reports the headline token-level numbers. We repeat the structure here with the per-model breakdown to show that the $\sim 10\%$-flag pattern is consistent across all four models, not a GPT-4o artefact. The entries are drawn from `analyze_token_focus.py` run on the calibration benchmark (144 entries: 104 HIGH, 40 LOW, four models).

Model	**Fn-flag rate**	**Tokens** > 0.30	**Avg fn size**
HIGH prompts ($n = 104$ across models)			
GPT-4o	96%	9.8%	97 tok
GPT-4o-mini	95%	10.2%	99 tok
GPT-4-turbo	97%	11.1%	96 tok
DeepSeek V3	99%	11.0%	102 tok
aggregate	**97.1**%	**10.5**%	**98** tok
LOW prompts ($n = 40$ across models)			
GPT-4o	90%	12.6%	78 tok
GPT-4o-mini	80%	12.4%	76 tok
GPT-4-turbo	90%	13.5%	82 tok
DeepSeek V3	100%	13.8%	80 tok
aggregate	**90.0**%	**13.1**%	**79** tok

Table 4: Function-level flag rate vs. token-level focus, broken out by model and prompt category. The function-level rate saturates; the token-level rate is stable at $\sim 10\%$ (flag) across models and across HIGH/LOW categories, which means the localization property holds whether the function-level fire is a true Type 2 flag or a Type 3 false-positive floor. Per-model counts have $n = 20$–30 entries each; numbers are rounded.

The aggregate is summarized in Table 3. Figure 2 (in §3.6) shows the per-token trajectory on one representative function; Figure 1 (in §1) stages the same example as the paper's motivating problem statement — the fluent surface, the hidden top-k split, and the 10-of-98-token review surface — in three panels.

4.2 Qualitative scenarios

Five scenarios exercise distinct failure modes. Scenarios A–D are single-run and cover four models (GPT-4o-2024-08-06, GPT-4o-mini, GPT-4-turbo, DeepSeek V3) with `logprobs=True, top_logprobs=5`, temperature = 0, and a system prompt specifying concise naming conventions. Scenario E is evaluated on GPT-4o across 16 independent runs to characterize within-model variation; single-run results for all four models on the other scenarios are in the multi-model table below.

		ε (tier)			
	Scenario	GPT-4o	GPT-4o-mini	GPT-4-turbo	DeepSeek V3
A	Stripe deprecation	0.878 F	0.665 F	0.680 F	0.775 F
B	OpenAI SDK v0 vs v1	0.560 F	0.487 F	0.678 F	0.646 F
C	SQLAlchemy 1.x vs 2.0	0.907 F	**0.000** C	0.911 F	0.925 F
D	FastAPI async/sync	0.505 F	0.506 F	0.774 F	0.679 F

Table 5: Scenarios A–D across four models. F = FLAGGED ($0.30 \leq \varepsilon < 0.95$); C = COMPLETE ($\varepsilon < 0.30$). 15 of 16 cells are FLAGGED; the lone miss is GPT-4o-mini on SQLAlchemy, where the model committed to one version with no hesitation ($\varepsilon = 0.000$) — a genuine false negative. No HIGH prompt crossed the PAUSED threshold ($\varepsilon \geq 0.95$) in this sweep.

A — Deprecated API (Stripe). *"Charge a customer's card for $50."* GPT-4o: $\varepsilon = 0.878$, FLAGGED. The peak token is the method selector after `stripe`: `.Ch` (`Charge`, prob. 0.52) vs. `.Payment` (`PaymentIntent`, prob. 0.47). Both paths are in the training data; neither is definitively preferred. All four models flag. The within-function review surface localizes to the method-selector region rather than the full 98-token body.

B — Confident wrongness (OpenAI SDK). *"Call GPT-4 and return the response text."* GPT-4o: $\varepsilon = 0.560$, FLAGGED. The model generated v0 (`openai.ChatCompletion.create(...)`) confidently; v0 raises `AttributeError` on SDK ≥ 1.0. ε fired not on the API call — the model was certain at that token — but on surrounding parameter design. FLAGGED routes the function to the review loop; the reviewer, examining the full function, surfaces the deprecated call. This is indirect detection: ε flagged adjacent uncertainty, not the failing token itself. All four models flag.

C — Syntax split (SQLAlchemy). *"Query all active users."* GPT-4o: $\varepsilon = 0.907$, FLAGGED, near the top of the FLAGGED range. GPT-4-turbo and DeepSeek V3 produce similarly high ε (0.911, 0.925). GPT-4o-mini, however, produces $\varepsilon = 0.000$ (COMPLETE): it commits to one version with no visible hesitation. This is not a typo or a data-collection artefact; it is real measured behaviour, and it is a genuine false negative. GPT-4o-mini committed to one SQLAlchemy version with complete confidence — no probability mass on the alternative — which means ε has nothing to observe regardless of whether the chosen branch is correct. The cross-model contrast is the interesting finding: intra-generational signal exists only when the model itself is uncertain, and training-data convergence on one branch silently removes it. This is the "confident wrongness" limitation of §6.1 made empirically concrete: ε cannot detect a model whose prior has converged on a particular API, no matter which branch the prior points to.

D — Compatibility mismatch (FastAPI async/sync). *"FastAPI endpoint that fetches weather from an external API."* GPT-4o: $\varepsilon = 0.505$, FLAGGED (solidly mid-range); GPT-4-turbo is

highest at 0.774. The model generated `async def` with `requests.get()` inside — two individually defensible decisions that combine into a silent bug (blocks the event loop). The cross-decision mismatch is not directly what ε measures: ε sees token-level uncertainty, which in this scenario fires on surrounding choices rather than on the async/sync combination itself. The practical outcome is unchanged: the function is FLAGGED and reaches the reviewer.

E — Multi-function module. Six auth functions with four interdependent decisions (password hash, JWT, SQLAlchemy, datetime). FLAGGED on all 16 runs, $\varepsilon \in [0.69, 0.92]$. Library choices varied across runs (passlib, bcrypt, PyJWT, python-jose, mixed), but each run looked internally confident. Early functions $\varepsilon = 0.40$–0.58; later functions frequently $\varepsilon = 0.000$ — an *uncertainty collapse* after initial commitment. The first function is the high-risk window.

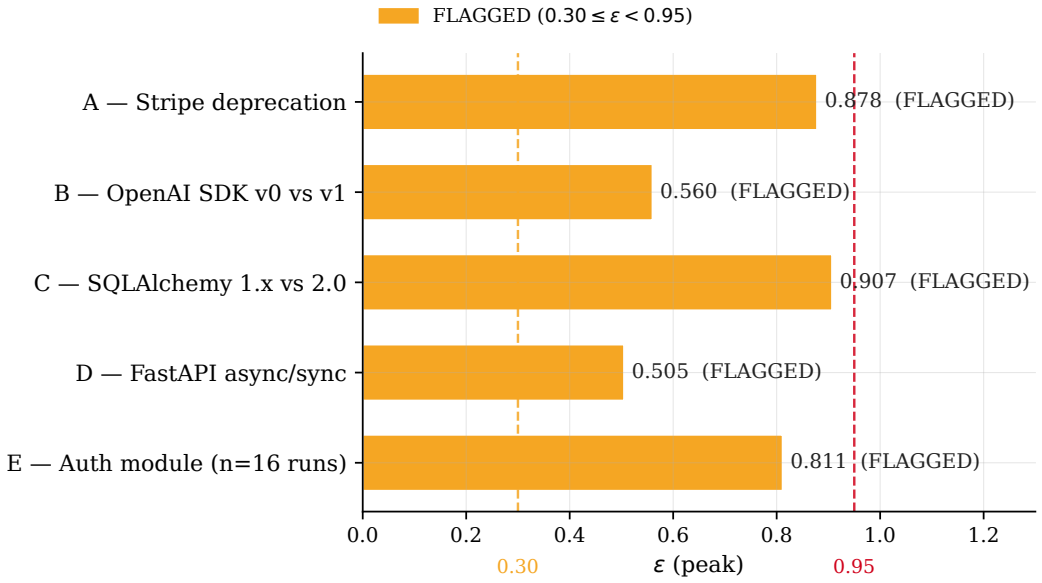

Figure 3: Scenarios A–E (GPT-4o) summarised by failure class and peak ε. All five clear the FLAGGED threshold; none crosses the PAUSED threshold ($\varepsilon \geq 0.95$).

4.3 Scenario E limitation: joint-prompt uncertainty collapse

Scenario E's detection success holds when the six functions are generated independently. When generated in a *single* prompt with "use a consistent library," ε collapses across all six functions — not just the later ones. The joint signature list plus the consistency instruction resolves the library choice before decoding starts, leaving nothing for ε to observe (Table 6, GPT-4o-mini). The practical mitigation is to generate functions independently; the consistency constraint can be applied at review time rather than at generation time.

GPT-4o is absent from Table 6 because on the combined-prompt variant it generates additional helper functions between the six expected signatures, making the positional chunk-to-function alignment used for per-function ε extraction unreliable. The 16-run variability study uses GPT-4o on independently-generated functions where alignment is not an issue; the collapse comparison uses GPT-4o-mini and DeepSeek V3, both of which produce the six expected functions without interstitial helpers.

Function	GPT-4o-mini		DeepSeek V3	
	Combined	Per-fn	Combined	Per-fn
`register_user`	0.579	0.627	0.424	0.864
`login`	0.820	0.872	0.518	0.972
`verify_token`	0.606	0.911	0.372	0.735
`refresh_token`	0.414	0.711	0.503	0.957
`request_password_reset`	**0.295**	0.788	0.764	0.760
`reset_password`	0.394	0.728	0.419	0.878

Table 6: Combined generation reduces per-function ε in both models. GPT-4o-mini's `request_password_reset` drops below the FLAGGED threshold (**0.295**, COMPLETE); DeepSeek V3 combined values stay above 0.30 but are uniformly lower than per-function values. When context resolves uncertainty before decoding, there is less for ε to observe.

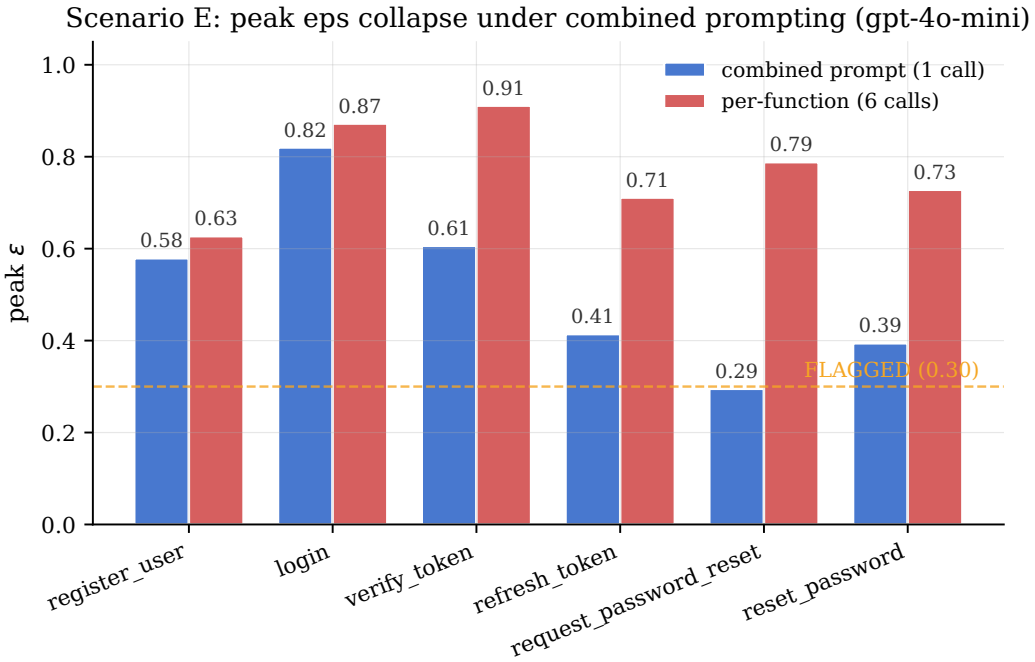

Figure 4: GPT-4o-mini: per-function generation keeps all six functions in the FLAGGED tier; combined generation collapses ε for every function, dropping one of six (`request_password_reset`, $\varepsilon = 0.295$) to COMPLETE. DeepSeek V3 shows the same reduction in combined-mode ε, but all six remain FLAGGED (Table 6).

4.4 Multi-model calibration

Moving from the per-scenario view to aggregate calibration, we evaluate a 30-prompt set: 10 LOW prompts (sort, palindrome, fibonacci, two-sum — pure logic) and 20 HIGH prompts (API / version-split tasks across Stripe, OpenAI, SQLAlchemy, FastAPI, Pydantic), run on four models. GPT-4o and DeepSeek V3 both achieve 85% HIGH detection; GPT-4o-mini and GPT-4-turbo both achieve 70%. DeepSeek V3 carries a higher LOW FP rate (70%) than the OpenAI models, consistent with its larger vocabulary of naming alternatives.

Model	HIGH detection	LOW FP
GPT-4o	85% (17/20)	50% (5/10)
GPT-4o-mini	70% (14/20)	60% (6/10)
GPT-4-turbo	70% (14/20)	60% (6/10)
DeepSeek V3	85% (17/20)	70% (7/10)

Table 7: Calibration at the FLAGGED threshold ($\varepsilon \geq 0.30$). 95% Wilson CIs: ±17pp HIGH, ±30pp LOW. Three HIGH prompts produce $\varepsilon = 0.000$ across all four models (`HTTP-GET-to-JSON`, `Pydantic serialization`, `async HTTP GET`) — *confidently-resolved domains* where training priors have converged.

4.5 low ground truth

All 30 LOW functions passed `exec()` assertions. Tier breakdown: COMPLETE 5, FLAGGED 20, PAUSED 5. The 25 non-COMPLETE fires are all Type 3 false positives: correct code where ε observed hesitation between algorithm structures that are both correct (brute vs. hash two-sum, recursive vs. iterative Fibonacci). Per §3.2, these are classified Type 3 precisely because both alternatives are correct; the decoder state that produced them is indistinguishable from the Type 2 state that fires on `Charge` vs. `PaymentIntent`.

4.6 Precision/recall at tier boundaries

We now consolidate the calibration numbers into a single precision/ recall view. On a 200-entry combined set (30 LOW exec-verified plus 170 HIGH LLM-judged; see §6.1), 53 true failures and 147 correct:

Threshold	Precision	Recall	F1
$\varepsilon \geq 0.30$ (FLAGGED+)	0.272	**1.000**	0.428
$\varepsilon \geq 0.95$ (PAUSED)	**0.309**	0.547	0.394

Table 8: $n = 200$, 53 positives; 95% Wilson CI on FLAGGED+ precision $[0.22, 0.33]$. Base failure rate 26.5% ≈ FLAGGED+ precision at the function level. The function-level gate's precision is barely above base rate; its value is recall, which is what directs the reviewer to the right functions, combined with the token-level localization of §3.6, which directs the reviewer to the right tokens within those functions. HIGH ground truth for 170/200 entries is LLM-judged by GPT-4o (§6.1); LOW ground truth (30/200) is `exec()`-verified. Recall = 1.000 is an observed result on this set; the known false negative (GPT-4o-mini on SQLAlchemy, $\varepsilon = 0.000$) falls outside this evaluated set and is discussed in §6.1. F1 is the harmonic mean $2PR/(P + R)$.

Function-level tiers are urgency signals, not calibrated probabilities. The useful binary boundary is COMPLETE vs. FLAGGED+. What actually makes the system work at the developer's end is that FLAGGED+ comes paired with a token-level attribution (§3.6).

4.7 Review loop: context-free vs. context-primed reviewer

Having established stage-1 recall and token-level localization, we turn to the stage-2 reviewer that converts the recall-maximizing filter into an end-to-end system with usable precision. The reviewer is given the function plus the specific high-ε tokens, and asked to evaluate those tokens in context. At current API prices this costs fractions of a cent per entry and completes in seconds.

Evaluated on 201 entries (production FastAPI codebase plus scenarios A–E, three models). Every FLAGGED+ entry is sent in parallel to a GPT-4o reviewer returning CLEARED/CONFIRMED; a consolidation pass processes remaining suspects. The **context-free** variant uses no reviewer context; the **context-primed** variant injects model-specific learned false-positive patterns from a prior clean run (injecting suspect patterns caused over-flagging in pilots). All results use GPT-4o as reviewer; reviewer-model ablation (e.g., with Claude or DeepSeek) is left to future work.

Metric	Context-free	Context-primed
Cleared by reviewer	107/201 (53.2%)	138/201 (68.7%)
Forwarded to consolidation	89	57
Confirmed true failures	85 (42.3%)	57 (28.4%)
Late false positives	4	0

Table 9: "Late false positive" = an entry marked CONFIRMED that was in fact correct on manual inspection. The context-primed variant is the preferred production configuration.

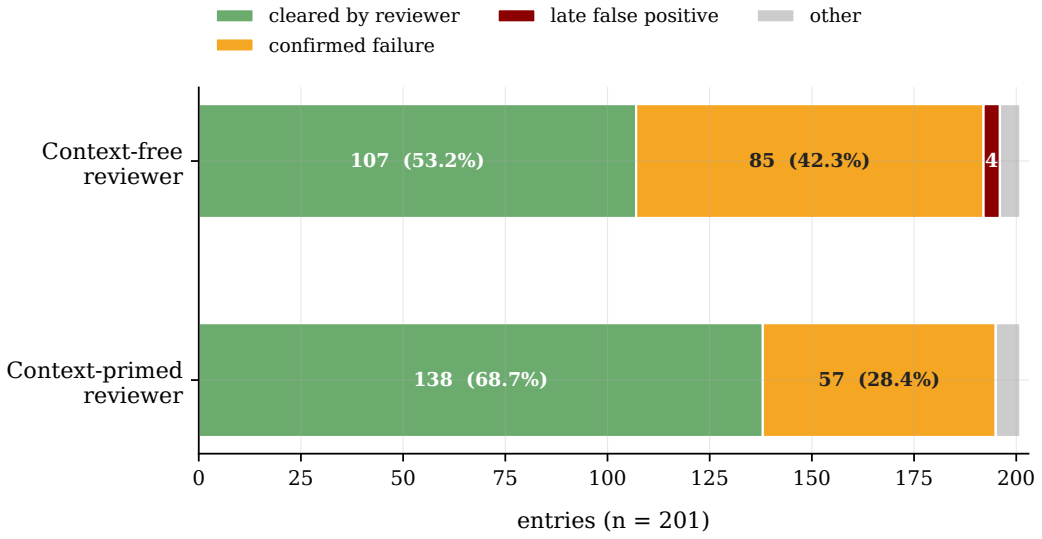

Figure 5: Injecting learned FP patterns into the reviewer (context-primed vs. context-free) shifts 31 additional entries from consolidation to cleared and eliminates late false positives, at the cost of suppressing some true failures the context-free reviewer forwarded.

End-to-end failure rates: GPT-4o-mini 40%, GPT-4o 29%, DeepSeek V3 26%. Because stage-1 recall is recall-maximizing (zero observed misses on this set), end-to-end precision is determined primarily by the reviewer's precision on the FLAGGED+ set; the reviewer's own precision is in turn driven by how well it can evaluate the specific tokens the filter attributes. The context-primed variant's zero late-FP result rests on $n = 57$ confirmed entries and needs a larger validation study. The stage-1 recall guarantee applies to entries that reach the reviewer; entries the reviewer clears were not independently verified.

5 Comparison with Alternative Approaches

The evaluation in §4 establishes that ε fires on the prompts it is designed to catch and that its token-level focus narrows the within-function review surface by 89%. What it does not establish is whether ε is the right signal relative to alternatives that read the same raw logprob data, or to alternatives that use the same underlying model via a different protocol. This section provides that comparison. The organizing point is simple: the function-level question — "which functions need scrutiny?" — is answered roughly equally well by ε, by p_{avg} [21], and (much worse) by multi-sample methods. The token-level question — "where within the function?" — is answered only by ε.

5.1 Signal taxonomy

Three fundamentally different uncertainty signals are in play. **Intra-generational token-level entropy** (ε; AdaDec) reads the top-k distribution at each decoding step and preserves per-token scores. **Sequence-level aggregate probability** (p_{avg}; the Spiess et al. [21] signal) averages log-probabilities over all generated tokens to produce a single scalar per function. **Inter-generational behavioural diversity** (multi-sample methods, UQLM [3]) runs the model multiple times at nonzero temperature and measures output-level agreement.

These are not three implementations of the same idea. The first two happen to read the same raw logprob data at the same 1× API cost, but aggregate it differently: ε localizes to a single high-entropy token after syntactic filtering, while p_{avg} smooths over the entire sequence. The third requires multiple model calls and measures a fundamentally different quantity: it cannot observe internal distributions at all, only the outputs those distributions produce.

5.2 p_{avg}: credit where due, and the distinguishing gap

We compute p_{avg} directly from the `mean_logprob` fields in the same calibration runs used to produce Table 7: $p_{\text{avg}} = \exp(\text{mean_logprob})$, with higher values indicating higher confidence. For a fair comparison, we choose the p_{avg} decision threshold on each model so that the resulting LOW false-positive rate matches ε's rate on that model; we then report the HIGH detection rate achieved at that operating point.

	ε (FLAGGED+)		p_{avg} (matched FP)	
Model	HIGH det.	LOW FP	HIGH det.	LOW FP
GPT-4o	85% (17/20)	50%	85% (17/20)	50%
GPT-4o-mini	70% (14/20)	60%	75% (15/20)	10%
GPT-4-turbo	70% (14/20)	60%	90% (18/20)	40%
DeepSeek V3	85% (17/20)	70%	65% (13/20)	10%

Table 10: ε vs. p_{avg} across four models at the function level. p_{avg} ties ε on GPT-4o and outperforms ε on GPT-4o-mini and GPT-4-turbo on raw HIGH detection; on DeepSeek V3, ε is higher (85% vs. 65%). At the same 1× API cost, the raw function-level detection picture favours neither signal uniformly. HIGH det. here is the flagging rate on HIGH-category prompts, not recall on true failures: when the model generates correct code for a HIGH prompt, ε correctly produces COMPLETE (low ε), and those entries lower this rate without representing missed failures.

p_{avg}'s real strength, stated plainly. On three of four OpenAI models, p_{avg} matches or beats ε's HIGH detection rate at the same or lower LOW false-positive rate, using the same API call and

the same logprobs. On GPT-4-turbo it is 90% vs. ε's 70% at a lower FP rate. This is not a small finding, and we do not want to bury it. For a user who only needs a function-level score and is working with the OpenAI API, p_{avg} is a strong, cheap, publicly-documented baseline.

Where p_{avg} stops, and ε continues. p_{avg} produces one scalar per function. It cannot tell a reviewer *where in the function* the uncertainty lives. On the Stripe example in Figure 6, p_{avg} reports a single number for the 98-token function; ε reports the same kind of function-level score *plus* an identification of the tokens whose per-token entropy drove it. The reviewer does not need to re-read the function — they need to evaluate the method-selector decision (see Table 3).

This is the distinguishing claim. It is not a higher rate; it is a different kind of output. We ran the p_{avg} baseline through the full review loop to measure whether that attribution gap produces a precision gap at the system level.

p_{avg} through the review loop. We re-ran all six production and scenario result files with `mean_logprob` captured, swept p_{avg} thresholds to find the one whose API-entry recall best matches ε's 93%, and ran the identical review loop without providing token attribution to the reviewer. The threshold sweep surfaces a structural constraint first: at every threshold, p_{avg}'s maximum recall on API entries is 44% (76/174). Even at the most aggressive threshold, `mean_logprob` stays near zero across production completions — the model is globally confident, and averaging over the full token sequence absorbs the localized uncertainty that ε reads at the decision token.

At the matched threshold (`-mean_logprob` ≥ 0.05), p_{avg} flags 99 entries. Through the review loop, end-to-end precision is 50.5% (50/99 confirmed true failures), compared to ε's 94% on 200 entries. All 50 suspect entries were confirmed by the consolidation pass (0 downgrades): the reviewer correctly identified issues when it found them, but cleared 49 genuine problems it could not localize to specific tokens.

Signal	Entries reviewed	API recall	End-to-end precision	True errors found
ε (cascaded)	200	93% (162/174)	**94**%	∼188
p_{avg} (≥0.05 threshold)	99	44% (76/174)	50.5%	50

Table 11: End-to-end comparison: ε vs. p_{avg} through the full review loop on the same 228-entry production-plus-scenario set (three models). p_{avg}'s recall ceiling is structural: `mean_logprob` averages over all tokens including confident boilerplate, absorbing localized API-decision uncertainty. The precision gap (43.5pp) reflects the reviewer's inability to localize without token attribution; errors were confirmed when found but 49 genuine failures were cleared because the reviewer had no token target to examine.

The system-level conclusion differs from the signal-level one. Token attribution is not a refinement on top of the filter; it is the mechanism that enables 94% end-to-end precision. Without it, the same review loop achieves 50.5%.

`min_logprob` as a sanity floor. The older single-token `min_logprob` baseline (Table 12) is complementary to both signals. It looks at the lowest top-1 probability in the generation, which misses near-tie tokens (e.g., 0.48/0.42): the top-1 is still 0.48 and the threshold does not fire. ε sees near-uniform top-k entropy on the same token and does.

Method	HIGH det. @ 30% LOW FP	HIGH det. @ 50% LOW FP
`min_logprob`	45–55%	55–65%
ε (max, AST-filtered)	**70**%	**85**%

Table 12: Matched LOW FP budgets across four models, single top-1 probability baseline. Gap is at near-tie tokens: `min_logprob` sees 0.48 and does not fire; ε sees near-uniform top-k entropy and does.

5.3 Multi-sample: structural blindspot

Inter-generational methods measure disagreement across multiple outputs of the same prompt. We ran each of the four models $N = 5$ times at temperature $T = 0.7$ on the full 30-prompt calibration benchmark, and compute output diversity by extracting the API-choice classification for each sample (e.g., `Charge` vs. `PaymentIntent` on Stripe) and measuring the normalized Shannon entropy over the choice distribution.

	ε (FLAGGED+)		Multi-sample N=5, T=0.7	
Model	HIGH det.	LOW FP	HIGH det.	LOW FP
GPT-4o	85% (17/20)	50%	20% (4/20)	10%
GPT-4o-mini	70% (14/20)	60%	10% (2/20)	30%
GPT-4-turbo	70% (14/20)	60%	25% (5/20)	30%
DeepSeek V3	85% (17/20)	70%	35% (7/20)	30%

Table 13: ε vs. multi-sample across four models. Multi-sample costs 5× as many API calls yet achieves 10–35% HIGH detection — 40–55pp below ε at comparable or worse LOW FP rates.

The reason multi-sample underperforms is not the sample count ($N = 5$) or the temperature ($T = 0.7$); it is structural. A model with probability 0.52 on `Charge` and 0.47 on `PaymentIntent` samples `Charge` on roughly 52% of re-rolls at $T = 1.0$, and nearly always at $T = 0.7$, where the sharpening exponent concentrates mass on the peak. The Stripe Scenario A example is a direct instance:

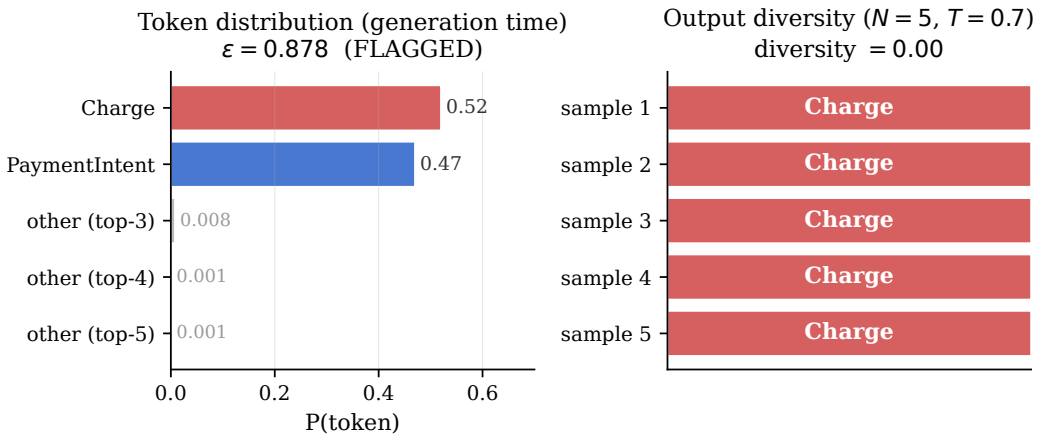

Figure 6: The intra/inter distinction made concrete on Stripe Scenario A (GPT-4o). Left: the token distribution at the method-selector token shows a near-tie ($P(\texttt{Charge}) = 0.52$, $P(\texttt{PaymentIntent}) = 0.47$), giving $\varepsilon = 0.878$ at that token. Right: five independent samples at $T = 0.7$ all emit `Charge` (diversity = 0.00). Multi-sample can only observe the right panel; the left panel is what ε reads. The 52/47 internal hesitation never surfaces as output variation.

Figure 6 is the empirical proof of the intra/inter distinction, not a theoretical assertion. GPT-4o with $\varepsilon = 0.878$ on Stripe chose `Charge` unanimously across all five samples. A stronger multi-sample check would not recover the signal: there is no output variation for a semantic-equivalence test or symbolic executor to disagree about. Our multi-sample measurement is output-level (which API was chosen); stronger equivalence checks [19] that compare code behaviour up to semantic equivalence would help on Type 3 false positives, where two semantically equivalent implementations are wrongly counted as disagreement, but they cannot recover intra-generational API-choice uncertainty that produces unanimous output.

UQLM [3] is the closest published off-the-shelf tool in this family; its `BlackBoxUQ` method samples each prompt 5 times and scores semantic disagreement via a DeBERTa NLI model. On our benchmark it detects 5% of HIGH-risk prompts at 0% LOW false-positive rate — a result driven by the same structural limit: the model is behaviourally consistent even when internally uncertain.

5.4 Static analysis

Static analysis is complementary to ε and is treated in §2.6. The method requires only top-k log-probabilities — no model-internal access — available via OpenAI's `logprobs` parameter, DeepSeek via Together AI, and vLLM for open-source models. Anthropic's API does not currently expose per-token log-probabilities.

5.5 Summary

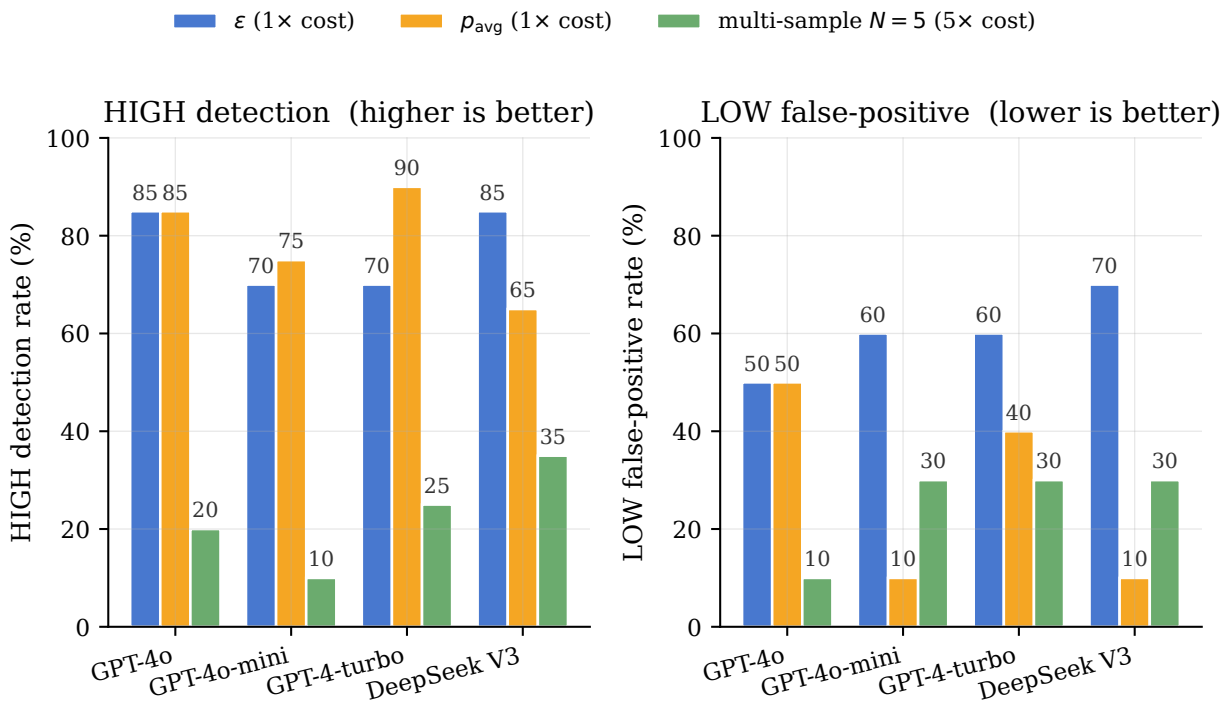

Figure 7: Three-method comparison across four models. Left: HIGH detection rate (higher is better). Right: LOW false-positive rate (lower is better). ε and p_{avg} operate at 1× API cost; multi-sample at $N = 5$ costs 5×. p_{avg} is competitive with ε on raw function-level detection; multi-sample is structurally blind to intra-generational uncertainty.

Through the full review loop (§4.7), the token-attribution gap becomes a precision gap: ε achieves 94% end-to-end precision; p_{avg}-routed review, without token localization, achieves 50.5% on the same infrastructure (Table 11).

6 Discussion

6.1 Limitations

Confident wrongness. ε catches *hesitation*, not *wrongness*. The two overlap but are not identical: a model that has converged on a deprecated API with sharp probability mass is wrong without being uncertain, and ε is silent there. ε cannot detect a model that is wrong and confident. If training placed all mass on a deprecated API, the token probability is sharp and ε is low; the system emits the wrong code with COMPLETE status. Scenario B is the concrete example: GPT-4o emitted `openai.ChatCompletion.create(...)` — the v0 API, which raises `AttributeError` on SDK ≥ 1.0 — with no hesitation at that token; ε fired only because of adjacent uncertainty in surrounding parameter design. Scenario C on GPT-4o-mini ($\varepsilon = 0.000$, §4.2) is the cleaner empirical instance: the model's prior on SQLAlchemy had converged on a single version, so there is no token-level hesitation for ε to see. Wang et al. [24] report this failure mode at scale: 25–38% deprecated-API emission on standard prompts across seven LLMs, driven by confident predictions

from training data that predates the deprecation. This is a fundamental limit of any generation-time signal and a primary motivation for pairing ε with static analysis (§5.4) rather than replacing it.

Confidently-resolved domains. Prompts where every model has converged on a strong training-data prior produce $\varepsilon = 0.000$ regardless of correctness. Current-UTC datetime retrieval and HTTP-GET-to-JSON fall here across all four models. For stable widely-deployed APIs this is desirable; for deprecated-but-confidently-recalled APIs it is Scenario B's failure mode.

Token-level localization depends on function-level fire. The token-level focus story (§3.6) rests on the function being FLAGGED in the first place. A confident-wrong function has no high-ε tokens by definition; the token-level claim does not rescue the function-level miss, because there is no miss to rescue — the system produced COMPLETE and the developer never sees a reviewer call. Token localization sharpens the review of already-flagged functions; it does not expand recall.

Small-sample calibration. Main calibration is 30 prompts per model; Wilson intervals on HIGH detection are ±17pp at $n = 20$. We claim recall-first design plus an empirical miss rate of zero on the sets evaluated, not a proof of perfect recall. The p_{avg} comparison in §5.2 uses the same 30 prompts, which bounds all statistical claims in that subsection to the same small-sample regime. The token-focus analysis aggregates 144 entries (four models, 30 prompts with some drop-outs) and should be read as characterising order of magnitude rather than a precise percentage.

p_{avg} comparison scope. The signal-level p_{avg} evaluation (§5.2) uses the calibration benchmark (30 prompts per model, 120 total). The end-to-end comparison through the review loop (§5.2, Table 11) uses the full 228-entry production-plus-scenario set. The recall ceiling finding — p_{avg}'s maximum API-entry recall is 44% at any threshold — rests on the production set and is not subject to the small-sample caveat that bounds the calibration results.

Multi-sample sensitivity. The multi-sample comparison uses $N = 5$ at $T = 0.7$. Higher N or higher T might recover additional behavioural diversity on some prompts — a $T = 1.0$ run would nearly double the sampling probability of the minority branch on a 0.52/0.47 token — though the structural argument does not depend on parameter choice: ε is read at $T = 0$ where the model's internal distribution is exposed directly, while any sampling-based method is limited to what that distribution produces as behaviour.

Type 3 floor. The Type 3 false-positive floor remains; the mitigation is reviewer speed (both branches pass).

LLM-judged ground truth. LOW ground truth is `exec()`-verified against assertions covering return type, edge cases, and correctness. HIGH ground truth is LLM-judged via GPT-4o. Circularity exists where the evaluated model shares a family with the judge (GPT-4o-mini judged by GPT-4o); results for that model should be read with that caveat. We disclose rather than overclaim human verification we did not perform.

Hyperparameters. The cascaded tier algorithm (six checks, three thresholds) was calibrated on GPT-4o; other models inherit these. The binary FLAGGED+ gate is robust; internal tier placement is a refinement on top.

6.2 paused tier and delivery semantics

§3.5 defined PAUSED as $\varepsilon \geq 0.95$ at the function level. Whether the function-level PAUSED tier actually holds code delivery until the review completes, or simply runs the review loop like FLAGGED, is an implementation choice we do not prescribe. Production use would likely default to the non-blocking treatment (same routing as FLAGGED) while exposing PAUSED as a higher-urgency signal in whatever UI surfaces reviewer output.

6.3 Context learning as a microcosm

Injecting learned false-positive patterns into the reviewer's context (the configuration evaluated in §4.7) raised clearance from 53.2% to 68.7%, and late false positives fell from 4 to 0. This is a trade-off, not a free win: the same priming that eliminates late false positives also suppresses some genuine failures that the context-free reviewer had forwarded (Figure 5). Practitioners in high-stakes settings should weigh the suppressed-failure cost against the clearance and late-FP gains. This is in-context improvement on a fixed model; training-time improvement would be expected to compound with it. The mechanism generalizes to any setting where a downstream model is judging upstream output with known failure patterns.

6.4 Future work

Per-token intervention. The token-level focus invites editor-level surfacing: highlight the handful of high-ε tokens inline as the developer reviews, the way a spellchecker does. A targeted re-decode at lower temperature on just the flagged span (AdaDec-style, but at review time) is a natural follow-up.

Token-routed multi-sample. Multi-sample is structurally blind at the function level but could recover signal if restricted to the high-ε token span at high temperature — a hybrid that uses ε to localize and sampling to explore. At $\sim 10\%$ of tokens this costs far less than full multi-sample.

Pipeline propagation. Upstream ε should propagate as prior uncertainty into downstream generations, modulating sampling temperature or review threshold.

Open-source models. vLLM exposes full top-k; a Llama / Qwen / Mistral calibration run would test whether the recall-first boundary and the 10.5% token-focus regularity hold outside OpenAI.

Cross-decision consistency. Scenario D's mismatches argue for AST-level consistency checks over ε (e.g., `async def` + blocking HTTP client).

Import-gated activation. Restricting ε scoring to functions that import external libraries would eliminate most of the Type 3 floor at negligible cost.

7 Conclusion

We introduced ε, a per-token normalized Shannon entropy over top-k log-probabilities, AST-filtered and max-aggregated into a per-function score that also preserves per-token attribution. As a recall-maximizing filter, the function-level claim on its own is not distinguishing. The sequence-level mean token probability p_{avg} of Spiess et al. [21] reads the same logprobs, costs the same $1\times$ in

API budget, and is competitive with or better than ε on function-level detection for three of four OpenAI models.

The distinguishing contribution is what the function-level score gives up to become a single scalar, and what ε preserves instead. A reviewer given ε's output is not asked to re-read the function; they are asked to evaluate the $\sim 10\%$ of semantic tokens that cross the flag threshold. No sequence-level or inter-generational signal produces this localization. Prior token-level work (ASTrust [16], Vasconcelos et al. [23]) shares the goal of token-level surfacing but is presented as standalone visualization or interface design; ε's contribution is to package the same kind of token-level signal as the routing layer of an end-to-end filter-plus-reviewer system. p_{avg} cannot: it is one scalar per function. Multi-sample cannot: a 0.52/0.47 internal split produces a unanimous output, as GPT-4o on Stripe Scenario A empirically demonstrates ($\varepsilon = 0.878$, sample diversity $= 0.00$ across $N = 5$ at $T = 0.7$). UQLM detects 5% of our HIGH prompts on the same benchmark. The intra-generational signal is a different quantity from the inter-generational one, and it is the quantity that happens to be actionable for code.

The two-stage architecture is how this becomes a working system. Stage 1 is the recall-maximizing filter; stage 2 is a parallel LLM reviewer directed to the specific high-ε tokens rather than asked to re-read the function. On 201 entries across three models, a context-primed reviewer clears 68.7% of flagged entries with zero late false positives.

The same mechanism — early-token commitment forcing later tokens, leaving the genuine uncertainty concentrated in a $\sim 10\%$ slice of the generation — plausibly applies in other structured-output domains (SQL, configuration files, typed program synthesis); empirical verification is left to future work. Generation is fast; review is the bottleneck. A recall-maximizing filter, plus a reviewer that is told exactly where to look, moves review off the developer's critical path.

References

[1] Anastasios N Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. Conformal risk control. In *International Conference on Learning Representations (ICLR)*, 2024.

[2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models, 2021.

[3] Dylan Bouchard, Mohit Singh Chauhan, David Skarbrevik, Ho-Kyeong Ra, Viren Bajaj, and Zeya Ahmad. UQLM: A Python Package for Uncertainty Quantification in Large Language Models. *Journal of Machine Learning Research*, 27(13):1–10, 2026.

[4] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. CodeT: Code generation with generated tests, 2022.

[5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code, 2021.

[6] Andrew G. A. Correa and Ana C. H. de Matos. Entropy-guided loop: Achieving reasoning through uncertainty-aware generation, 2025. arXiv:2509.00079.

[7] David Gros and Prem Devanbu. Localized calibrated uncertainty in code language models, 2025. arXiv:2512.24560.

[8] Neha Gupta, Harikrishna Narasimhan, Wittawat Jitkrittum, Ankit Singh Rawat, Aditya Krishna Menon, and Sanjiv Kumar. Language model cascades: Token-level uncertainty and beyond, 2024. arXiv:2404.10136.

[9] Kaifeng He, Mingwei Liu, Chong Wang, Zike Li, Yanlin Wang, Xin Peng, and Zibin Zheng. Towards better code generation: Adaptive decoding with uncertainty guidance, 2025.

[10] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[11] Alain Jungo, Raphael Meier, David Alleysson, and Mauricio Reyes. Analyzing the quality and challenges of uncertainty estimations for brain tumor segmentation with deep neural networks. *Frontiers in Neuroscience*, 2020.

[12] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. Language models (mostly) know what they know, 2022.

[13] Haziq Mohammad Khalid, Athikash Jeyaganthan, Timothy Do, Yicheng Fu, Sean O'Brien, Vasu Sharma, and Kevin Zhu. ERGO: Entropy-guided resetting for generation optimization in multi-turn language models. In *Proceedings of the 2nd Workshop on Uncertainty Aware NLP (UncertaiNLP 2025)*, 2025. arXiv:2510.14077.

[14] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *International Conference on Learning Representations (ICLR)*, 2023.

[15] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback, 2023. NeurIPS 2023; arXiv:2303.17651.

[16] David N. Palacio, Daniel Rodriguez-Cardenas, Alejandro Velasco, Dipin Khati, Kevin Moran, and Denys Poshyvanyk. Towards more trustworthy and interpretable LLMs for code through syntax-grounded explanations. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 2024. Under review.

[17] Victor Quach, Adam Fisch, Tal Schuster, Adam Yala, Jae Ho Sohn, Tommi S Jaakkola, and Regina Barzilay. Conformal language modeling. In *International Conference on Learning Representations (ICLR)*, 2024.

[18] Andrea Santilli, Adam Golinski, Michael Kirchhof, Federico Danieli, Arno Blaas, Miao Xiong, Luca Zappella, and Sinead Williamson. Revisiting uncertainty quantification evaluation in language models: Spurious interactions with response length bias results. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. arXiv:2504.13677.

[19] Arindam Sharma and Cristina David. Assessing correctness in LLM-based code generation via uncertainty estimation, 2025.

[20] Yewei Song, Tiezhu Sun, Xunzhu Tang, Prateek Rajput, Tegawende F. Bissyande, and Jacques Klein. Measuring LLM code generation stability via structural entropy. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering, New Ideas and Emerging Results (ASE-NIER)*, 2025.

[21] Claudio Spiess, David Gros, Kunal Suresh Pai, Michael Pradel, Md Rafiqul Islam Rabin, Amin Alipour, Susmit Jha, Prem Devanbu, and Toufique Ahmed. Calibration and correctness of language models for code. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025. arXiv:2402.02047.

[22] Neeraj Varshney, Wenlin Yao, Hongming Zhang, Jianshu Chen, and Dong Yu. A stitch in time saves nine: Detecting and mitigating hallucinations of LLMs by validating low-confidence generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2024.

[23] Helena Vasconcelos, Gagan Bansal, Adam Fourney, Q. Vera Liao, and Jennifer Wortman Vaughan. Generation probabilities are not enough: Uncertainty highlighting in AI code completions. *ACM Transactions on Computer-Human Interaction (TOCHI)*, 2024.

[24] Chong Wang, Kaifeng Huang, Jian Zhang, Yebo Feng, Lyuye Zhang, Yang Liu, and Xin Peng. LLMs meet library evolution: Evaluating deprecated API usage in LLM-based code completion. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025. arXiv:2406.09834.

[25] Beining Xu and Yongming Lu. TECP: Token-entropy conformal prediction for LLMs, 2025. arXiv:2509.00461.

[26] He Ye, Matias Martinez, and Martin Monperrus. SelfAPR: Self-supervised program repair with test execution diagnostics. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2022.